TrustLayer on DCP

Scoring speech deepfakes across a network of browsers and machines



genuine



synthetic

Why this matters

3 seconds

of recorded speech was enough for Microsoft's VALL-E to imitate a new voice (2023)

2015

first ASVspoof challenge. The anti-spoofing benchmark has run roughly every two years since.

4.04 s

of audio is all this detector looks at for one verdict, so long recordings mean many runs

Cloned voices are cheap to make. Checking them has to happen wherever audio arrives, at the volume it arrives.

- Phone fraud that imitates a family member or an executive.
- Newsrooms and platforms checking clips before they spread.
- Researchers re-scoring whole corpora every time a model, a noise condition or an attack changes.
- Each clip is independent of every other clip, so the work splits cleanly across machines.

Whose work this is

The detector

AASIST: audio anti-spoofing using integrated spectro-temporal graph attention. Jee-weon Jung and colleagues, NAVER Clova, ICASSP 2022. Pretrained weights, MIT licence.

The camp project

TrustLayer, a QMIND Camp team: Reyan, Jinpeng, Kyle, Daniel, Shreya and Abdullah, advised by Shaina Raza. They wrapped the detector, tested it under noise and built a comparison screen.

The data

ASVspoof 2019 logical access: bona fide speech plus clips from 2019-era text-to-speech and voice-conversion systems. Showcase samples from the Yamagishi lab at NII.

What we added

ONNX conversion, a work function that does the whole pipeline in the DCP sandbox, a Node dispatcher, a web page that deploys the job, and the tests that tie them together.

How AASIST tells real speech from synthetic

4.04 s of audio
64,600 samples at 16 kHz

→

Sinc filter bank
70 fixed band-pass filters

→

Residual encoder
6 convolution blocks

→

Two graphs
one over frequency, one over time

→

Graph attention
joins the two, in two branches

→

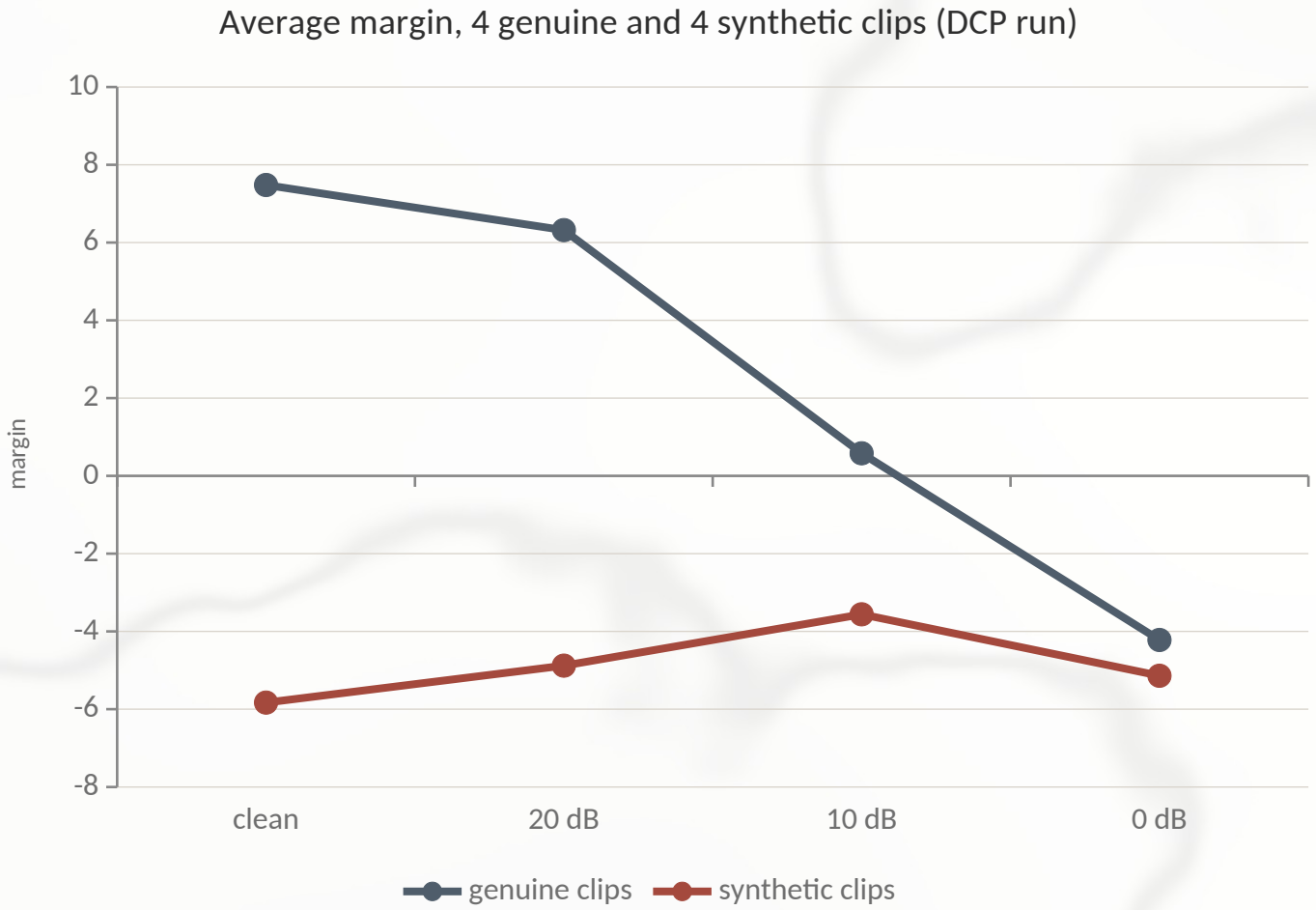
Two scores
synthetic, genuine



margin = genuine score − synthetic score

- Above zero, the model calls the clip genuine; below zero, synthetic. The size is confidence of a sort, not a probability.
- About 300,000 weights, 1.3 MB. Small enough to send with every job.
- Trained on ASVspoof 2019 only, so it knows 2019-era synthesis, not today's voice-cloning services.

The camp question: does background noise fool it



The team's experiment

- 8 showcase clips, 4 genuine and 4 synthetic.
- Each clean and with white noise at 20, 10 and 0 dB signal-to-noise.
- Noise drags genuine speech down toward the boundary. At 0 dB, every genuine clip is called synthetic.
- Synthetic clips stay synthetic: the detector fails in one direction, false alarms.

One model file any worker can run

1.63 MB

ONNX model, exported from the 1.28 MB PyTorch weights. About 2.2 MB as base64 inside a job.

3×10^{-6}

largest difference from PyTorch, over 32 test windows (8 clips × 4 noise levels)

0

verdicts changed by the conversion, with optimisation on or off, one clip at a time or 32 at once

What made it convert cleanly

- The sinc filter bank is fixed, not learned, so it becomes one constant convolution.
- The graph pooling keeps a fixed number of nodes because the input length is fixed, so the only dynamic dimension is the batch.
- All 34 operation types in the graph (convolution, attention, TopK, scatter) are supported by ONNX Runtime's WebAssembly backend, which every DCP worker can run.

The client packs the files; the workers do the rest

Your machine

- Reads each audio file as it is: WAV or FLAC, any rate.
- Encodes it as text (base64).
- Groups a few files per slice.
- That's all. Milliseconds per file.

DCP

Each DCP worker

1. **Decode** WAV (PCM or float) and FLAC, written in plain JavaScript
2. **Mix and resample** to mono at 16 kHz, the same filter as scipy's resample_poly
3. **Window** 4.04 s windows, sliding every second across long clips
4. **Add noise** white noise at each chosen SNR, seeded so any worker gives the same result
5. **Run the model** batches of windows through ONNX Runtime (WebAssembly)
6. **Report** a margin per window, and a verdict per file

Doing this on the client would make one machine the bottleneck for the whole corpus. Here it scales with the number of workers.

How the job is laid out

Input set

changes every slice

- A few raw audio files per slice, as base64.
- Each with a name and, if known, a label.
- Files per slice chosen so a slice runs 10 to 30 seconds.

Static arguments

shared by every slice

- The model itself, 2.2 MB as base64. Swap in another checkpoint without changing code.
- The plan: noise levels, seeds, window hop.
- Options: WebAssembly or WebGPU, batch size, how to combine windows.

Work function

the pipeline

- One self-contained function, so a browser page can deploy it.
- Its only dependency is DCP's published ONNX Runtime package.
- Keeps the loaded model between slices on the same worker.

Same numbers on every runtime

Runtime	Largest margin difference	Time per 4 s window
Python + native ONNX Runtime (reference)	0	0.3 s
Node + ONNX Runtime WebAssembly (local harness)	3×10^{-6}	0.57 s
Chrome tab (same function, tested in the page)	same function, same results	0.6 s
DCP worker (native Mac worker, WebAssembly)	4×10^{-5} (CSV rounding)	0.8 s

A bug in the decoder or the resampler would show up as a score difference, so one end-to-end check covers all of them.

23 test files

run through the work function and through the original Python loader, compared window by window

- WAV: 16-bit, 32-bit float
- FLAC: 16-bit mono, 24-bit stereo
- 44.1 kHz and 22.05 kHz sources
- a 12-second clip with 9 windows

A web page that launches the job



Results

Margin = genuine logit − synthetic logit (a raw score, not a probability). The verdict uses the lowest-margin window, so one synthetic-looking window flags the file.

10	88	52 s	30/36
files scored	model runs	worker inference time	correct (labelled rows)

CONDITION	GENUINE KEPT GENUINE	SYNTHETIC CAUGHT	ALL
clean	5/5	4/4	9/9
20 dB	5/5	4/4	9/9
10 dB	4/5	4/4	8/9
0 dB	0/5	4/4	4/9

Per file

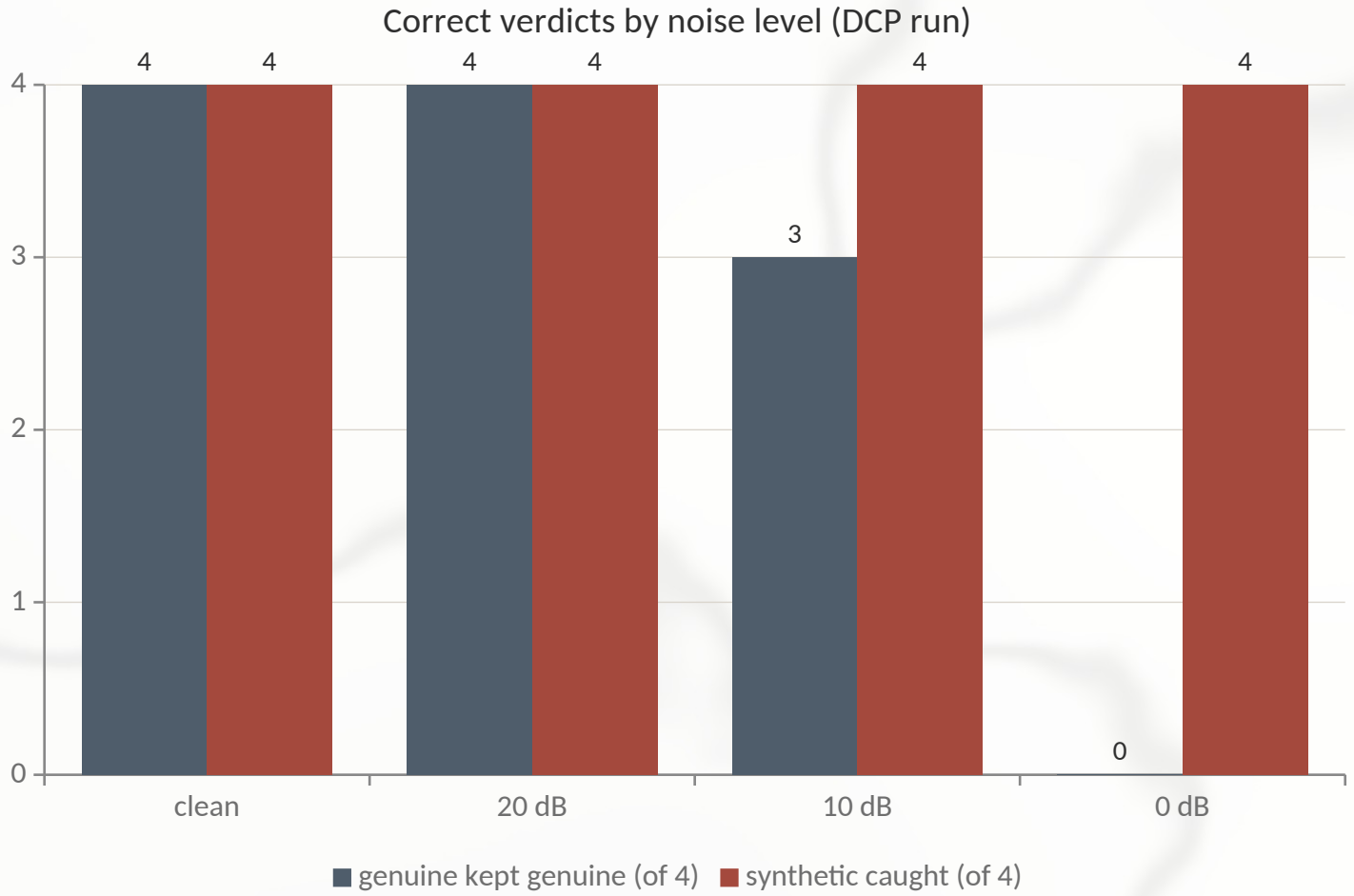
Each line is one condition; the points are 4-second windows across the clip. Below the dashed zero line, the model leans synthetic.

clean — 20 dB — 10 dB — 0 dB

FILE	LABEL	CLEAN	20 DB	10 DB	0 DB	WINDOWS
genuine_1.wav wav, 16000 Hz, 4.741 s	genuine	genuine 7.19	genuine 5.54	genuine 1.63	synthetic −4.00	
genuine_2.wav wav, 16000 Hz, 3.119 s	genuine	genuine 8.66	genuine 7.74	genuine 0.90	synthetic −3.20	
genuine_3.wav wav, 16000 Hz, 3.63 s	genuine	genuine 7.48	genuine 6.61	genuine 1.31	synthetic −4.35	
genuine_4.wav wav, 16000 Hz, 5.335 s	genuine	genuine 5.68	genuine 4.34	synthetic −1.50	synthetic −5.32	
synthetic_1.wav wav, 16000 Hz, 2.5 s	synthetic	synthetic −4.18	synthetic −4.35	synthetic −2.37	synthetic −4.42	
synthetic_2.wav wav, 16000 Hz, 2.26 s	synthetic	synthetic −11.93	synthetic −9.24	synthetic −3.67	synthetic −4.51	
synthetic_3.wav wav, 16000 Hz, 2.525 s	synthetic	synthetic −6.68	synthetic −4.87	synthetic −3.61	synthetic −6.56	
synthetic_4.wav wav, 16000 Hz, 1.794 s	synthetic	synthetic −0.52	synthetic −1.06	synthetic −4.60	synthetic −5.05	
spliced_g_s_g.flac flac, 16000 Hz, 11.982 s	–	genuine 1.13	genuine 0.67	synthetic −1.38	synthetic −5.10	
genuine_1_44100.flac flac, 44100 Hz, 4.741 s	genuine	genuine 7.21	genuine 6.71	genuine 1.62	synthetic −3.92	

- Open dcp.dev/projects/trustlayer. Nothing to install.
- Paste an API key, pick compute groups and a payment offer.
- Drop WAV or FLAC files, or load the sample clips.
- Choose noise levels, seeds and how far windows slide.
- Deploy: slices stream back with a live log and progress bar.
- Results by condition, a verdict per file, and a timeline of every window. Download as CSV or JSON.
- A QR code beside the log opens a browser worker in the same compute group, so anyone in the room can help run the job.

Noise turns real voices "synthetic"



0 of 4

genuine clips still called genuine at 0 dB, noise as loud as the voice

4 of 4

synthetic clips still caught at 0 dB

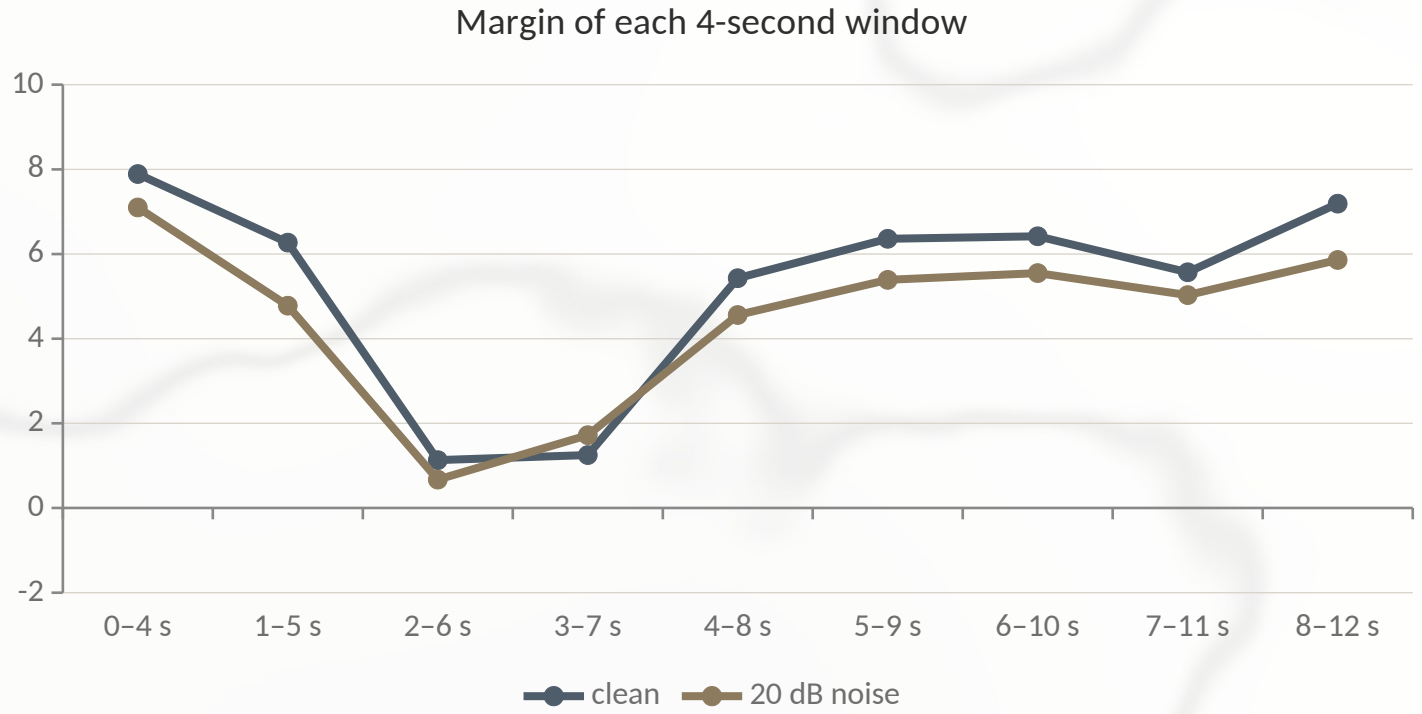
Eight hand-picked clips show the direction of the effect, not the detector's accuracy in the wild.

A fake spliced into real speech slips through



+1.13

lowest window margin, clean: still above zero, so the whole file is called genuine



- The lowest windows (2–6 s, 3–7 s) straddle the splice point at 4.7 s. The window that is mostly synthetic (4–8 s) still scores about +5.
- AASIST was trained on whole clips, each entirely real or entirely fake. It was never taught to find a fake inside a real one.
- Models trained on partly faked audio (the PartialSpoof data) score every 20 ms and mark the edits.

Stronger detectors fit the same job

AASIST (today)

- 300 thousand weights, 1.6 MB
- Learned from 2019-era speech synthesis
- One verdict per 4-second window
- About 1 s per window on a CPU worker

Speech encoder + AASIST

- A large pretrained speech model (wav2vec 2.0 / XLS-R) feeding a small AASIST-style classifier
- About 300 million weights: 1.2 GB, or about 300 MB at 8-bit
- Clearly better on real-world recordings in published results
- Much more compute per clip, which is where DCP pays off

Partial-fake localisation

- Same kind of speech encoder, trained on partly faked audio
- Scores every 20 ms and marks real-to-fake boundaries
- Answers both "is it fake?" and "where was it edited?"
- Same windowed job, finer output

The job layout doesn't change: swap the model in the static arguments, keep the decoding, windowing and noise.

What a full benchmark run looks like

71,237

clips in the ASVspoof 2019 logical-access evaluation set

≈ 16 h

of one worker's time for a single clean pass, at 0.8 s per window and about one window per clip

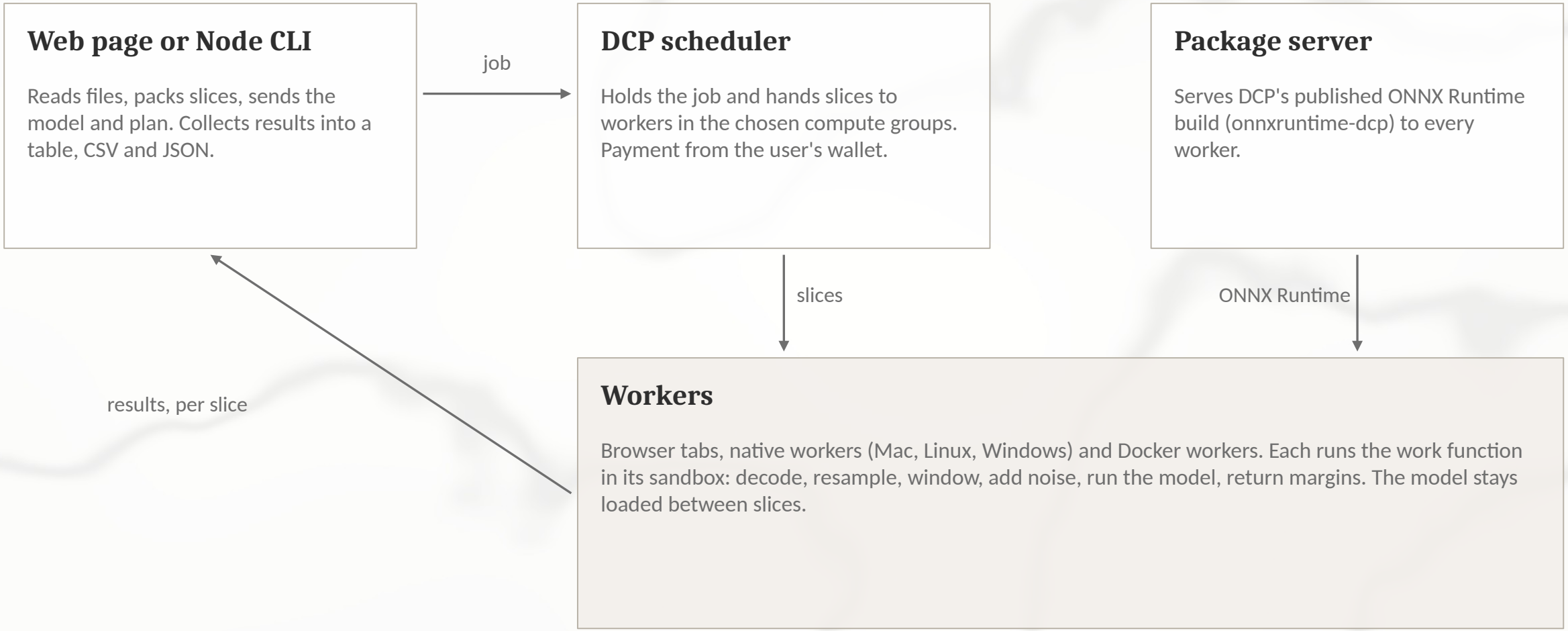
≈ 10 min

on 100 workers, and every extra noise level or seed multiplies the work, not the wait

Next steps

- Run jobs from dcp.dev/projects/trustlayer on the Queen's compute group and the public network.
- Run the full ASVspoof 2019 evaluation set: clean, then a noise sweep with several seeds.
- Add an MP3 decoder for recordings as they usually arrive, and time WebGPU against WebAssembly.
- Convert a speech-encoder detector and a partial-fake localiser, and run them through the same job.

Architecture



Try it

Open dcp.dev/projects/trustlayer, drop in some audio, choose your noise levels, deploy.

